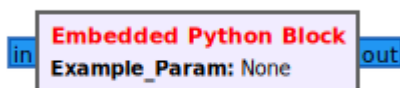


Python Blocks

There are two ways to add custom python blocks within gnuradio: Out of tree modules and embedded blocks. Examples of out of tree modules include the SNL repositories that you are familiar with and installed from source, `gr-pdu_utils`, `gr-sandia_utils`, `gr-timing_utils`, `gr-fhss_utils`. This method for installing python blocks is the more comprehensive route, but also requires a bit more effort. We will first discuss embedded blocks, which are available natively to gnuradio. In contrast, these are essentially "already installed", but due to this exhibit some slightly finicky behavior that makes them less suitable for sharing across repos and for collaborating. They are however excellent for testing out ideas or debugging flowgraphs.

Embedded Python Blocks

An [embedded python block](#) can be added to a gnuradio flowgraph by searching for "Python Block". By default, embedded blocks are handled to process complex streaming data, indicated by the blue input/output ports on the block.



Once this has been dragged into the flowgraph, you can double click on it to open the source code from the editor of your choice. It should look identical to the following screen shot

```

2 Embedded Python Blocks:
3
4 Each time this file is saved, GRC will instantiate the first class it finds
5 to get ports and parameters of your block. The arguments to __init__ will
6 be the parameters. All of them are required to have default values!
7 """
8
9 import numpy as np
10 from gnuradio import gr
11
12
13 class blk(gr.sync_block): # other base classes are basic_block, decim_block, interp_block
14     """Embedded Python Block example - a simple multiply const"""
15
16     def __init__(self, example_param=1.0): # only default arguments here
17         """arguments to this function show up as parameters in GRC"""
18         gr.sync_block.__init__(
19             self,
20             name='Embedded Python Block', # will show up in GRC
21             in_sig=[np.complex64],
22             out_sig=[np.complex64]
23         )
24         # if an attribute with the same name as a parameter is found,
25         # a callback is registered (properties work, too).
26         self.example_param = example_param
27
28     def work(self, input_items, output_items):
29         """example: multiply with constant"""
30         output_items[0][:] = input_items[0] * self.example_param
31         return len(output_items[0])
32

```

The python class itself is derived from `gr.sync_block`. Sync blocks are designed for blocks that consume and produce an equal number of items per port (1 value out for every 1 value in). In contrast, there are other classes, such as `gr.decim_block` or `gr.interp_block` which are used for decimating or interpolating, respectively. Lastly, there is the more abstract `gr.basic_block` which does not assert any relationship between the number of inputs and outputs and is the most general form. More information about the differences can be found [here](#).

The `__init__` method allows you to configure several key pieces of information:

- User-defined parameters that are passed via the GUI `example_param=1.0` which can then be saved as member variables for later use.
- The input and output ports. Since these have the below settings, we know that both the input and output ports will accept/return complex streaming data.
 - `in_sig=[np.complex64]`
 - `out_sig=[np.complex64]`

Streaming Blocks

For blocks that deal with streaming data, `work` will be the bread and butter for getting any DSP modifications added to your data. In the default example, this block is scaling the streaming data by the value set in `example_param`.

Exercises

- Write a streaming block that takes in two float streams and combines them into a single complex output.
- Write a streaming block that decimates an incoming signal by a user-defined integer amount
- Write a streaming block that outputs the phase of incoming IQ data.
- Write a streaming block that calculates the power of an incoming stream and tags it whenever the power exceeds a user-defined threshold.
- Write a streaming block that applies a low pass filter to an incoming signal with user-defined taps.

PDU Blocks

PDU's are the essential data type we use in gnuradio since we primarily deal with bursty radio modems. PDU's are comprised of two pieces of information, a meta dictionary and data. You can identify when PDU's are being passed between GR blocks by the grey input/output ports.

Unlike streaming data, blocks that accept PDU's as inputs do not use the work block. Instead, they use callback functions, which are functions that are called every time a PDU is passed through an input port. Below is an extremely simple example that takes an incoming PDU and adds a key to the PDU's meta data, whose value is determined by a user-defined value. When a PDU arrives at the **Input** port, line 30 is telling the block to pass that PDU to the member function `add_key_value`. Recall that PDU's use GR's internal PMT types and this information must be converted to/from python data types. A list of these conversions can be found [here](#).

```

1  """
2  Embedded Python Blocks:
3
4  Each time this file is saved, GRC will instantiate the first class it finds
5  to get ports and parameters of your block. The arguments to __init__ will
6  be the parameters. All of them are required to have default values!
7  """
8
9  import numpy as np
10 from gnuradio import gr
11 import pmt
12
13 class blk(gr.basic_block): # other base classes are basic_block, decim_block, interp_block
14     """Embedded Python Block example - a simple multiply const"""
15
16     def __init__(self, key = 'foo', value=1.0): # only default arguments here
17         """arguments to this function show up as parameters in GRC"""
18         gr.sync_block.__init__(
19             self,
20             name='Add Key to PDU', # will show up in GRC
21             in_sig=None,
22             out_sig=None
23         )
24         # if an attribute with the same name as a parameter is found,
25         # a callback is registered (properties work, too).
26         self.key = key
27         self.value = value
28
29         self.message_port_register_in(pmt.intern("Input"))
30         self.set_msg_handler(pmt.intern("Input"), self.add_key_value)
31
32         self.message_port_register_out(pmt.intern("Output"))
33
34     def add_key_value(self, pdu):
35         meta = pmt.car(pdu)
36         data = pmt.cdr(pdu)
37         meta = pmt.dict_add(meta, pmt.intern(self.key), pmt.from_double(self.value))
38         self.message_port_pub(pmt.intern("Output"), pmt.cons(meta, data))
39
40

```

Notice the following main differences between PDU blocks and Streaming blocks

- `import pmt` on line 11
- The values of `in_sig` and `out_sig`
- Manually defining input and output ports via `self.message_port_register_in` and `self.message_port_register_out`
- Defining the callback function for the input ports on line 30
- Manually publishing data in line 38.

Exercises

- Write a block that takes a PDU and splits the PDU into two output ports, one containing the meta data dictionary and one containing the data.
- Write a block that takes a PDU and only passes it as output if it has a corresponding user-defined key/value pair.
- Use your block from Exercise 3 of streaming data make a second block that reads the tags from streaming data for when a burst starts and stops and converts that into PDUs. Be sure to include relevant information as meta data in the output PDU.

- Write a block(s) that adds AWGN to a burst and calculates the SNR of the signal and incorporates this information in the metadata. Make sure that the amount of noise added is a user-defined parameter.
- Write a block that takes in IQ streaming data and converts every N entries into a PDU.
- Write a block that dewhitens bursts from an RFD900x

Big Exercises

- Combine some of the above exercises to write a series of blocks that takes incoming streaming data, tags the signals where bursts occur, and then converts those bursts to PDUs after being basebanded and filtered.

Out of Tree Modules

The content of the blocks themselves are the same, but you have to install these from source using [these instructions](#). The make/cmake instructions will vary depending on how you installed gnuradio, but editing the YAML files is the same.